

金融資料探勘期中報告

吳瑞庭

財金碩二 611336040

負責年度：2021

繳交日期：4/4

【目錄】

作業目的.....	2
原始資料說明.....	2
資料構建流程.....	2
最終索引檔欄位說明.....	3
成果展示.....	5
問題與修正.....	6
AI Prompt 實作與優化紀錄摘要.....	8
學習反思.....	9

【作業目的】

本實作採用 Python 語言於 Google Colab 雲端環境開發，旨在處理 2021 年度的金融原始數據，建立一套標準化的「選擇權分析索引檔」。透過自動化腳本串聯雲端硬碟（Google Drive）中的多份 CSV 資料來源，確保資料處理的精確性與可重複性。

【原始資料說明】

本次分析整合了三項關鍵數據源，奠定定價模型的基礎：

1. **標的交易資料**：提供 2021 年每日收盤價，作為 S_0 基準。
2. **到期結算資料**：包含台指選擇權月、週契約之最後結算日。
3. **銀行利率資料**：以台灣銀行一年期定儲機動利率，作為無風險利率 R_f 。

【資料建構流程】

1. 建立自動化檔案關聯 (File Mapping)

依據「年月日」欄位，透過字串格式化生成 file（作業）欄位（格式：OptionsDaily_YYYY_MM_DD.csv）。此舉打破了資料孤島，使主索引表能與每日數萬筆的選擇權報價明細檔（Raw Files）建立自動化連結。

2. 契約篩選與轉倉邏輯 (Contract & Roll-over)

為確保研究對象的流動性與代表性，建立了以下篩選機制：

- **排除週契約**：過濾掉契約名稱中含 "W" 的資料，僅保留標準月選擇權。
- **設定 Maturity ≥ 1 條件**：計算「最後結算日」與「交易日」之天數差距。若差距 ≥ 1 則標註為當月契約；若當日即為結算日（距離為 0），則自動滾動（Roll-over）至下一個月的月合約。此邏輯真實模擬了法人機構在結算日前的換倉行為。

3. 整合市場參數(R_f)

針對 2021 年全球低利環境進行參數校準：

- **數據鎖定**：對照台銀利率表，選取一年期機動利率。因 2021 年央行政策穩定，索引檔統一填入 0.84%，確保定價模型中的折現因子一致。

【資料建構流程】

1. 日期整理 (Date Cleaning)

這是資料處理的首要步驟。原始 CSV 檔案中的日期（如 2021/1/4）通常被識別為一般字串 (String)。透過 Python 的 `pd.to_datetime` 工具，將這些字串轉換為時間序列物件 (Datetime Object)。

- 目的：讓電腦「讀懂」時間，以便後續進行日期運算（如計算距離到期的天數）或進行時間排序。

2. 欄位改名 (Column Renaming)

為了確保程式碼在讀取不同來源（如交易資料、利率資料、到期日資料）時不會出錯，需統一欄位名稱。

- 做法：檢查原始檔案的標題 (Header)，將可能包含空格、全形符號或不一致的名稱（如「收盤價」與「收盤價(元)」）進行標準化。
- 目的：確保在合併多份表格時，程式能精確對應到正確的變數，避免產生空值。

3. 建立 File (作業)

根據作業規範與資料庫索引需求，將日期格式化為特定的檔名格式。

- 邏輯：將日期物件轉換為 `OptionsDaily_YYYY_MM_DD.csv` 字串。

例如：2021/01/04 → `OptionsDaily_2021_01_04.csv`。

- 用途：作為「路徑索引」，讓程式能自動定位並讀取該交易日對應的詳細選擇權日報檔。

4. 建立 Contract (契約月份)

從「到期日資料」中比對，找出當日應觀察的契約。

- 篩選邏輯：

(1)排除週契約：過濾掉含有 "W" 字樣的代碼，僅保留標準「月選擇權」。

(2)最近月判定：尋找距離當前交易日最近的到期契約，且必須符合

Maturity ≥ 1 的轉倉條件。

- 用途：明確標註該日分析的標的合約，確保資料具備連貫性。

5. 建立 ContractExpiryDate（最後結算日）

在決定了 Contract 之後，從到期日對照表中抓取該契約對應的「最後結算日」。

- 用途：作為計算合約存續期間的終止點，是財務定價模型中時間參數的基礎。

6. 建立 Maturity（剩餘到期天數）

這是計算出來的衍生欄位，公式為：最後結算日 - 交易當日日期。

- 關鍵邏輯：採納「大於等於 1 天」邏輯。若交易日當天就是結算日，則會自動滾動（Roll-over）至下一個月的契約，使 Maturity 重新起算。
- 用途：對應 Black-Scholes 模型中的時間參數 T。

7. 建立 Rf（無風險利率）

對照「銀行定儲利率資料」，將市場無風險利率填入索引檔。

- 數據來源：採用台灣銀行一年期定期儲蓄存款之機動利率。
- 實作細節：針對 2021 年的資料，由於該年度利率政策穩定，統一填入 0.84%。
- 用途：作為折現率，計算選擇權理論價值或進行敏感度分析（Greeks）。

【最終索引檔欄位定義與用途】

產出欄位	數據定義	在後續分析中之用途
file (作業)	報表索引標籤	作為讀取每日詳細選擇權報價檔（如 IV 計算）的自動化路徑。
Contract	契約月份代碼	區分時間序列中不同契約的存續期間。
Maturity	剩餘到期天數	決定定價模型中的時間參數 TS 。
Rf	無風險利率	計算理論價格與希臘字母的必要參數。

【成果展示】

2021/1/4	14,902.03	OptionsDaily_2021_01_04.csv	202101	2021/1/20	16	0.84
2021/1/5	15,000.03	OptionsDaily_2021_01_05.csv	202101	2021/1/20	15	0.84
2021/1/6	14,983.13	OptionsDaily_2021_01_06.csv	202101	2021/1/20	14	0.84
2021/1/7	15,214	OptionsDaily_2021_01_07.csv	202101	2021/1/20	13	0.84
2021/1/8	15,463.95	OptionsDaily_2021_01_08.csv	202101	2021/1/20	12	0.84

【問題與修正】

1. 日期格式整理 (Date Formatting)

- 問題描述：
 - 原始資料中的日期通常為字串格式（如 2021/1/4），無法直接進行加減運算。
 - 建立 file（作業）欄位時，需要特定的補零格式（如 01_04 而非 1_4）。
- 修正方式：
 - 使用 `pd.to_datetime()` 將字串轉換為 Python 的 Datetime 物件。
 - 利用 `.dt.strftime('OptionsDaily_%Y_%m_%d.csv')` 強制轉換為具備前導零的標準格式，確保與原始日報檔名完全匹配。

2. 欄位型態修正 (Column Types)

- 問題描述：
 - 金額欄位（如 收盤價(元)）常帶有千分位逗號（如 15,000.03），導致 Pandas 將其視為「字串」而非「數字」，無法進行 B-S 模型運算。
 - 部分欄位在讀取時可能產生空值 (NaN)。
- 修正方式：
 - **去逗號化**：使用 `.str.replace(',', '')` 移除逗號，再透過 `.astype(float)` 轉換為浮點數型態。
 - **空值處理**：使用 `.fillna()` 補全缺失資料（例如在 2021 年利率穩定期補入固定數值）。

3. 契約判定 (Contract Determination)

- 問題描述：
 - 「到期日資料」中混合了「月選擇權」與「週選擇權（含 W 字樣）」。

- 作業要求僅分析月選擇權，需排除流動性與性質不同的週契約。
- 修正方式：
 - 布林過濾：使用 `~df['契約月份'].str.contains('W')` 篩選掉所有包含 "W" 的字串。
 - 精確匹配：確保剩餘的契約代碼（如 202101）皆為純月合約。

4. 到期日對應與 Maturity 計算 (Expiry Mapping)

- 問題描述：
 - 轉倉時機：若交易日正好是結算日（天數為 0），是否應繼續沿用舊契約？
 - 搜尋效率：如何在大規模到期表中快速找到對應交易日的最近到期日？
- 修正方式：
 - 大於等於 1 邏輯：修正判定條件為 最後結算日 - 交易日 $\geq$ 1。若不滿足（即剩餘天數為 0），程式自動跳轉尋找下一期契約。
 - 向量化運算：在 Python 中建立對照函數，確保每一筆交易日都能精確對應到符合條件的最近一個結算日。

5. Rf（無風險利率）合併 (Rf Merging)

- 問題描述：
 - 「利率資料表」通常是分段更新，並非每日紀錄。
 - 2021 年以前的資料可能存在缺失值。
- 修正方式：
 - 區間對照：對照台灣銀行一年期定儲機動利率表。
 - 數值鎖定：經比對 2021 年市場狀況，確認央行該年度未調整利率，因此採取「常數補入法」，統一將 Rf 設為 0.84 (0.84%)，避免因表格關聯失敗產生的空值影響後續模型計算。

【 AI Prompt 實作與優化紀錄摘要】

專案實作：2021 台指選擇權 (TX0) 分析索引檔建立

1. 任務目標 (Objectives)

旨在透過 Python 自動化程序，整合分散的原始數據（標的價格、結算日、市場利率），建立一套具備金融邏輯的定價分析索引檔，為後續 B-S 模型運算提供精確參數。

2. 技術實作與優化 (Implementation & Optimization)

針對 Google Colab 執行環境，進行了以下技術迭代：

- **路徑自動化：**捨棄易出錯的手動路徑，改用 `os.walk` 遍歷雲端硬碟 (Google Drive)，自動偵測並讀取數據源，解決 `FileNotFoundError` 問題。
- **契約自動轉倉邏輯：**
 - **過濾機制：**排除週選擇權 (W)，僅鎖定標準月合約。
 - **轉倉判定 ($Maturity \geq 1$)：**計算交易日與結算日天數差。若剩餘天數小於 1，系統自動跳轉至次月合約，模擬實務上的平倉與換倉行為。
- **參數標準化：**統一日期補零格式 (如 01_04)，並針對 2021 年利率環境補全無風險利率 ($R_f = 0.84\%$)。

3. 實作成果與驗證 (Results)

透過程式自動化處理，成功產生 2021 年度全交易日索引。

- **驗證實例：**以 2021/01/20 結算日為例，系統成功判定 $Maturity=0$ ，並自動將後續交易日之契約從 202101 切換至 202102，確保了分析數據的連貫性與實務準確性。

【學習反思】

我認為 AI 在幫助我們寫程式的這方面確實有極大的幫助，但也確實出現嚴重的缺陷，我們人類能以肉眼判讀檔案名稱，但 AI 沒有辦法，他只能以完整檔名去搜尋檔案，因此指令不能下錯，如果要使用 AI 幫忙讀取檔案，最好的方法是去雲端硬碟上面複製完整檔案名稱，這樣 AI 在寫程式時比較不容易出錯。

另外 colab 的優點是在執行程式碼時，旁邊會有 AI 小助手，如果在 gemini 上面生成錯誤程式碼的話，可以透過旁邊的小助手幫忙進行優化，本次作業我有遇到尋找不到檔案的問題，也是透過旁邊的小助手，幫忙先搜尋到我要的檔案，再進行獨取檔案的動作。

目前 AI 相對於人類有明顯的優勢與劣勢，優勢在於可以完成大量複雜數據的資料整理及編寫程式碼，但劣勢也相對明顯，他沒有辦法相人類透過肉眼判讀檔案名稱的相似性，可能我想要的檔案跟資料夾裡面的檔案差一個數字或是底線的格式不同，就會造成在電腦判讀上出現錯誤，會導致程式碼出錯，因此我們在 debug 的部分上也會花費我們不少的時間。

總結來說我認為 AI 是一項好的工具，在於你如何下達指令，輸入的話完不完整，因此會對於表達能力差的人來說會顯的不好運用。